TP — Sécuriser le backend d'un gestionnaire de tâches

Spring Boot, Spring Security et client React Planora

Durée : **60 minutes**, après une présentation théorique de 30 minutes. Travail individuel ou en binôme. Technologies : Java 17, Maven, Spring Boot et React/Vite.

Nom et prénom : ……
Groupe : …… **Date :** ……

1. Le produit et le travail demandé

Planora est une application de gestion des tâches d'une équipe. Son interface React et son API métier sont déjà fournies : on peut consulter, créer, modifier, terminer et supprimer des tâches. Les statistiques proviennent des tâches réellement stockées par Spring.

Le TP fait évoluer **le backend Spring**, en ajoutant progressivement l'authentification puis les autorisations. Utilisez l'application pour constater les effets de vos changements. Les règles d'accès doivent être imposées par Spring ; aucune modification de React n'est demandée.

À la fin, vous saurez distinguer authentification et autorisation, déclarer des comptes avec BCrypt, configurer `SecurityFilterChain`, protéger une méthode avec `@PreAuthorize` et vérifier les parcours métier avec les tests et les réponses HTTP.

Les tâches sont partagées par l'équipe : **Alice et Bob peuvent gérer toutes les tâches**. Le TP ne met pas en place de propriété individuelle des tâches. Les statistiques et le projet confidentiel sont réservés à Bob, administrateur.

Contrat de l'API

Méthode et route	Fonction	Succès
`GET /api/public/info`	Information publique, à consulter avec curl	`200`, texte
`GET /api/tasks`	Liste des tâches	`200`, tableau JSON
`POST /api/tasks`	Création	`201`, tâche JSON
`PUT /api/tasks/{id}`	Modification	`200`, tâche JSON
`PATCH /api/tasks/{id}/complete`	Passage à l'état terminé	`200`, tâche JSON
`DELETE /api/tasks/{id}`	Suppression	`204`, sans corps
`GET /api/admin/stats`	Statistiques calculées	`200`, objet JSON
`GET /api/tasks/confidential`	Note du projet Orion, fournie à partir de la branche 5	`200`, objet JSON

Le CRUD et les statistiques existent dès la branche 1. La route confidentielle est absente des branches 1 à 4 ; elle est déjà implémentée dans la branche 5, où il reste à la protéger.

Une tâche contient `id`, `title`, `description`, `status`, `priority`, `dueDate` et `assignee`. Le formulaire React permet de modifier le titre, la description, l'état, la priorité et l'échéance. La personne assignée est affichée ; elle n'est pas éditable dans ce formulaire et le serveur la conserve lors d'une modification.

États : `TODO`, `IN_PROGRESS`, `DONE`. Priorités : `LOW`, `MEDIUM`, `HIGH`. Le titre est obligatoire et limité à 160 caractères ; la description à 1 000 caractères ; l'échéance utilise le format `AAAA-MM-JJ`. Des données invalides renvoient `400` et un identifiant de tâche inexistant renvoie `404`, si les filtres de sécurité laissent passer la requête.

Les données restent **en mémoire**. Un redémarrage de Spring restaure les trois tâches d'exemple. Au démarrage, les statistiques sont : 3 tâches, 1 terminée, 2 en attente, 1 de priorité haute et 33 % d'achèvement. Les statistiques d'administration sont calculées sur toutes les tâches ; les compteurs de l'écran des tâches utilisent la liste chargée dans React.

Architecture

Partie	Responsabilité
`frontend/src/main.jsx`	Écrans métier et formulaire de connexion
`frontend/src/api.js`	Appels JSON et en-tête HTTP Basic

Partie	Responsabilité
`controller/`	Routes HTTP et appels des services
`model/`	Objets échangés avec React
`service/TaskService.java`	CRUD et stockage en mémoire
`service/StatisticsService.java`	Indicateurs calculés
`config/UserConfig.java`	Comptes, rôles et encodeur BCrypt
`config/SecurityConfig.java`	Règles HTTP et sécurité des méthodes

Les chemins Java sont relatifs à `src/main/java/fr/tpspringsecurity/`. Les fichiers de configuration apparaissent au fil des branches. La même interface React est présente sur les six branches.

2. Comprendre les branches

Chaque branche est **le point de départ de son exercice**, avec les corrections précédentes. La branche 6 contient la solution finale. Les numéros ne désignent donc pas des exercices déjà terminés.

Branche	État initial	Travail demandé	Durée
`Tp_version1`	CRUD et statistiques ouverts	Auditer les accès	10 min
`Tp_version2`	API encore ouverte, dépendance à ajouter	Installer Spring Security	10 min
`Tp_version3`	Sécurité par défaut, compte généré	Déclarer Alice et Bob	15 min
`Tp_version4`	Alice et Bob présents, politique par défaut	Configurer les droits HTTP et CSRF	10 min
`Tp_version5`	Droits HTTP corrigés ; note lisible par tout compte valide	Activer la protection de méthode	10 min
`Tp_version6`	Sécurité complète	Vérifier le parcours final	5 min

Après l'exercice 2 et après l'exercice 3, les lectures demandent un compte valide, y compris l'information publique. Les rôles ne limitent pas encore les statistiques. **Sans jeton CSRF, les mutations renvoient `403`, même avec un compte valide et même sans compte.** L'exercice 4 adapte cette configuration pour permettre les mutations du client fourni.

Le compte généré `user` apparaît après l'ajout de la dépendance en exercice 2 et reste présent au début de la branche 3. Alice et Bob existent **après la réalisation de l'exercice 3**, et dès l'ouverture des branches 4 à 6.

3. Préparer et lancer le projet

Prérequis : JDK 17, Maven 3.6.3 ou supérieur, Git et Node.js **20.19+ ou 22.12+** avec npm. Vérifiez notamment que `mvn -version` utilise le JDK attendu.

```
java -version
mvn -version
node --version
npm --version
```

Récupérez le dépôt :

```
git clone https://github.com/romarich57/Tp_Spring_security.git
cd Tp_Spring_security
git switch Tp_version1
```

Dans un premier terminal, à la racine :

```
mvn test
mvn spring-boot:run
```

Dans un second terminal :

```
cd frontend
npm install
npm run dev
```

Ouvrez **http ://localhost :5173**. L'API écoute sur **http ://localhost :8080**. Vite relaie `/api` vers Spring, ce qui permet à React de travailler sur la même origine via le proxy.

Utiliser Planora

- **Mes tâches** : consulter, rechercher, filtrer, créer, modifier, terminer et supprimer.
- **Statistiques** : consulter les indicateurs du backend.
- **Projet confidentiel** : lire la note lorsque le backend propose cette route et autorise l'accès.
- **Connexion** : saisir le nom du compte et son mot de passe ; se déconnecter pour revenir au parcours anonyme.

Le formulaire transmet les identifiants en HTTP Basic à chaque requête. Ils restent en mémoire React, sans cookie ni stockage local ; recharger la page les efface. Il n'existe pas de route de login dédiée. L'affichage du nom saisi ne prouve pas que le compte est valide : Spring vérifie les identifiants à chaque appel et peut renvoyer `401`.

L'interface affiche les données et les erreurs réelles du serveur. Pour relever précisément un statut de succès, utilisez l'onglet Réseau du navigateur, `requests.http` ou `curl`. L'information publique n'a pas d'écran dans Planora : vérifiez-la avec `curl`.

Passer à la branche suivante

Arrêtez Spring et Vite avec `Ctrl+C`. Enregistrez les fichiers de votre exercice dans un commit local, puis changez de branche et relancez les deux serveurs. Exemple après l'exercice 2 :

```
git add pom.xml src/test/java/fr/tpspringsecurity/TaskApiSecurityTest.java
git commit -m "Exercice 2 terminé"
git switch Tp_version3
```

Les branches suivantes fournissent déjà les corrections précédentes ; il n'est pas nécessaire de reporter votre commit sur elles. Vous pouvez aussi mettre vos changements de côté avec `git stash push -m "Mon exercice"` ; ne réappliquez pas automatiquement ce stash sur la branche suivante.

4. Exercice 1 — Enquête : contrôle d'accès cassé (10 min)

Branche : `Tp_version1`. Dans cette première étape, ne corrigez pas le code : votre objectif est d'observer, de démontrer et d'expliquer.

Point de départ : OWASP Top 10

L'OWASP Top 10 :2025, A01 — Broken Access Control décrit les situations où une application laisse quelqu'un agir au-delà des permissions prévues. Parmi les exemples cités : une API qui n'applique pas de contrôle aux opérations `POST`, `PUT` ou `DELETE`, ou un accès à une fonction privilégiée en appelant directement son URL. Dans cet exercice, recherchez des indices de cette catégorie dans Planora. Vous devrez établir vous-même quels accès sont problématiques et proposer les règles à appliquer.

Mener l'enquête

1. Lancez Spring Boot et Planora. Restez **déconnecté**.
2. Dans **Mes tâches**, consultez les tâches, puis créez-en une, modifiez-la, terminez-la et supprimez-la. Notez ce que l'interface vous laisse faire.
3. Ouvrez **Statistiques**. La navigation dans React est un indice à relever, pas une preuve suffisante du contrôle d'accès.
4. Vérifiez les requêtes directement contre l'API : observez les appels dans l'onglet **Réseau** du navigateur ou utilisez `curl`. Testez les routes listées dans la fiche ci-dessous **sans identifiants**. Pour les opérations de modification, utilisez un identifiant de tâche existant ; supprimez uniquement la tâche que vous venez de créer.
5. Comparez ce que vous observez avec les permissions que vous estimez nécessaires pour une application d'équipe. Pour vous guider, regardez qui peut lire les tâches, modifier les données de l'équipe et consulter des statistiques d'administration.
6. Lisez `TaskController`, `AdminController`, `TaskService` et `StatisticsService`. Repérez où sont déclarées les routes et où se trouve (ou ne se trouve pas) une vérification d'accès.

L'interface publique `/api/public/info` peut être vérifiée avec `curl` ; elle n'a pas d'écran dédié dans Planora. Les réponses initiales de l'API sont observables ainsi :

```
curl -i http://localhost:8080/api/tasks
curl -i http://localhost:8080/api/admin/stats
curl -i http://localhost:8080/api/public/info
```

Ne vous contentez pas de dire qu'une opération « fonctionne » : gardez le statut HTTP, le résultat visible et l'action réalisée comme éléments de preuve.

Fiche d'observation à compléter

La fiche ci-dessous fait partie de ce support. Complétez-la dans le PDF (ou imprimez ces pages). Pour chaque requête, notez ce que **vous** avez observé sans vous connecter. Si une réponse diffère de ce que vous attendiez, expliquez pourquoi cela vous semble important.

Requête testée sans connexion	Accessible ?	Statut HTTP	Résultat ou effet observé
`GET /api/tasks`			
`POST /api/tasks`			
`PUT /api/tasks/{id}`			
`PATCH /api/tasks/{id}/complete`			
`DELETE /api/tasks/{id}`			
`GET /api/admin/stats`			

1. Décrire les permissions. Pour chacun des trois profils ci-dessous, indiquez les droits que l'application devrait accorder selon vous. Justifiez votre proposition.

- Une personne sans compte : _____
- Un membre connecté de l'équipe : _____
- Un administrateur : _____

2. Relier les observations à OWASP A01. Quelle différence avez-vous constatée entre les permissions que vous proposez et les accès réellement possibles ? Donnez au moins deux exemples précis, avec la requête et son résultat.

_____

_____

3. Expliquer le risque. Quelles données pourraient être exposées ou quelles actions pourraient être effectuées par une personne qui ne devrait pas y avoir accès ?

_____

_____

4. Distinguer deux notions. Avec vos mots, expliquez :

- Authentification : _____
- Autorisation : _____

5. Localiser le contrôle. Pourquoi cacher un bouton dans React ne suffirait-il pas à protéger la route correspondante ? Dans quelle partie du projet faudrait-il appliquer le contrôle d'accès ?

_____

_____

Conclusion — À partir de vos preuves, expliquez en quelques phrases pourquoi le cas observé correspond, ou non, à **A01 :2025 Broken Access Control**. La branche suivante introduira la première évolution du backend ; gardez votre fiche pour comparer les résultats après les exercices.

5. Exercice 2 — Installer Spring Security (10 min)

Branche : `Tp_version2`. **Fichier :** `pom.xml`.

1. Constatez que l'application est encore ouverte au début de cette branche.
2. Complétez le TODO de `pom.xml` avec la dépendance indiquée ci-dessous. Ne précisez pas de version : le parent Spring Boot la gère.
3. Redémarrez Spring et relevez le mot de passe généré dans sa console.
4. Sans compte, ouvrez les tâches et les statistiques : les lectures JSON renvoient maintenant `401`. Même `GET /api/public/info` est protégé.
5. Saisissez `user` et le mot de passe généré dans Connexion. Les lectures fonctionnent ; les mutations sans jeton CSRF renvoient `403`.

Dépendance : `spring-boot-starter-security`.
Groupe Maven : `org.springframework.boot`.

```
curl -i -u user:MOT_DE_PASSE_GENERE http://localhost:8080/api/tasks
```

Pourquoi le compte existe-t-il sans déclaration dans votre code ? Pourquoi l'information publique est-elle protégée ? Avez-vous déjà défini une règle `ADMIN` ?

Les tests d'intégration sont dans `TaskApiSecurityTest.java`, sous `src/test/java/fr/tpspringsecurity/`.

Activez le test suivant en retirant `@Disabled` :
`exercice2LesRoutesDemandentUneConnexion`.

Exécutez `mvn test`. Ce test vérifie les **lectures anonymes**, pas les mutations.

6. Exercice 3 — Déclarer les comptes avec BCrypt (15 min)

Branche : `Tp_version3`. **Fichier :** `config/UserConfig.java`.

Identifiant exact	Mot de passe local	Rôle
`alice`	`alice123`	`USER`
`bob`	`bob123`	`ADMIN`

Complétez les TODO 3.1 à 3.4 :

1. Déclarez un Bean `PasswordEncoder` retournant `BCryptPasswordEncoder`.
2. Déclarez un Bean `UserDetailsService` recevant cet encodeur.
3. Construisez Alice et Bob avec `User.withUsername(...)` et `.roles(...)`.
4. Encodez les mots de passe avec `encoder.encode(...)` et retournez un `InMemoryUserDetailsManager` contenant les deux comptes.

Ne modifiez pas les règles HTTP dans cet exercice. Redémarrez Spring, connectez-vous successivement avec les deux comptes et consultez les tâches et les statistiques. Essayez aussi un mauvais mot de passe.

Résultat attendu : Alice et Bob lisent les tâches **et les statistiques** ; un mauvais mot de passe produit `401`. Les rôles sont déclarés mais aucune règle ne réserve encore les statistiques à `ADMIN`. Les mutations restent refusées avec `403` sans jeton CSRF.

Expliquez la différence entre déclarer un rôle et imposer une autorisation, ainsi que l'intérêt de BCrypt. Retirez `@Disabled` du test `exercice3AliceEtBobPeuventSeConnecter`, puis lancez `mvn test`.

7. Exercice 4 — Configurer les accès HTTP (10 min)

Branche : `Tp_version4`. **Fichier :** `config/SecurityConfig.java`.

Les comptes sont déjà corrigés. Complétez les TODO 4.1 à 4.6 :

1. Déclarez un Bean `SecurityFilterChain` recevant `HttpSecurity`.
2. Autorisez `GET /api/public/info` avec `permitAll()`.
3. Réservez `GET /api/admin/**` à `hasRole("ADMIN")`.
4. Exigez `authenticated()` pour toutes les autres requêtes.
5. Activez HTTP Basic et `SessionCreationPolicy.STATELESS`.

6. Désactivez CSRF avec `.csrf(csrf -> csrf.disable())`, puis construisez la chaîne.

Déclarez les règles spécifiques avant `anyRequest()`. Les comptes restent dans `UserConfig`.

Choix pour ce TP local : le client fourni envoie explicitement l'en-tête `Authorization`, utilise `credentials: "omit"` et n'utilise pas de cookie de session. On désactive CSRF pour permettre ses mutations JSON. **Une API sans état ou utilisant HTTP Basic n'est pas automatiquement à l'abri de CSRF** : le navigateur peut envoyer automatiquement des identifiants Basic dans d'autres configurations. Si l'authentification ou le client change, il faut réévaluer cette politique. Voir la documentation Spring Security sur CSRF.

Vérifiez avec Planora :

— Anonyme : les tâches sont refusées avec `401`.
— Alice : création `201`, modification et clôture `200`, suppression `204` ; statistiques `403`.
— Bob : gestion des tâches et statistiques autorisées.
— Avec `curl`, `GET /api/public/info` fonctionne sans identifiants.

La route confidentielle est encore absente : un compte valide reçoit `404`, tandis qu'un appel anonyme est bloqué par la sécurité avec `401`.

Retirez `@Disabled` du test `exercice4LAccesPublicEtLesDroitsAdminSontRespectes`, puis lancez `mvn test`. Le test inclut une création de tâche authentifiée : oublier CSRF laisse cette opération en échec.

8. Exercice 5 — Protéger une méthode (10 min)

Branche : `Tp_version5`. **Fichiers :** `controller/TaskController.java` et `config/SecurityConfig.java`.

La route `GET /api/tasks/confidential` et son objet `ConfidentialProject` sont **déjà fournis**. Elle appelle `taskService.confidential()` et retourne le projet Orion, son client, un résumé et une note interne. Aucun endpoint métier n'est à créer.

1. Connectez-vous avec Alice et ouvrez Projet confidentiel : au début, elle peut lire la note (`200`). La règle HTTP générale exige seulement une authentification.
2. Activez `@PreAuthorize("hasRole('ADMIN')")` sur la méthode `confidential()` de `TaskController` (TODO 5.1).
3. Ajoutez `@EnableMethodSecurity` à `SecurityConfig` (TODO 5.2).
4. Redémarrez et vérifiez : anonyme `401`, Alice `403`, Bob `200`.

Ne créez pas de matcher HTTP spécifique pour cette route : l'exercice porte sur la sécurité des méthodes. `hasRole('ADMIN')` vérifie l'autorité `ROLE_ADMIN` créée par `.roles("ADMIN")`.

Expliquez le rôle de `@EnableMethodSecurity`. Comparez cette protection avec les règles HTTP. Pourquoi les données restent-elles dans le service ?

Retirez `@Disabled` du test `exercice5MethodeConfidentielleReserveeAuxAdmins`, puis lancez `mvn test`. Vérifiez que le CRUD et les statistiques fonctionnent toujours selon les règles de l'exercice 4.

9. Exercice 6 — Vérifier le parcours final (5 min)

Branche : `Tp_version6`. La correction complète est fournie ; aucun TODO de sécurité à compléter.

```
mvn clean verify
```

Le test d'intégration couvre l'accès public, les identifiants invalides, les accès aux statistiques et à la note, les mutations d'Alice, l'évolution des statistiques et la suppression d'une tâche inexistante. Les tests unitaires vérifient la logique métier. Tous les tests sont actifs en version 6.

La matrice suivante suppose des données valides et un identifiant de tâche existant :

Opération	Anonyme	Alice `USER`	Bob `ADMIN`
`GET /api/public/info`	200	200	200
`GET /api/tasks`	401	200	200
`POST /api/tasks`	401	201	201
`PUT /api/tasks/{id}`	401	200	200
`PATCH /api/tasks/{id}/complete`	401	200	200
`DELETE /api/tasks/{id}`	401	204	204
`GET /api/admin/stats`	401	403	200

Opération	Anonyme	Alice `USER`	Bob `ADMIN`
`GET /api/tasks/confidential`	`401`	`403`	`200`

Dans Planora, refaites un cycle complet de gestion des tâches avec Alice, puis vérifiez les statistiques et le projet confidentiel avec Bob. Recommencez ces accès avec Alice et sans compte. Utilisez curl ou l'onglet Réseau pour contrôler les statuts précis.

10. Aide et bilan

Symptôme	Vérification
API inaccessible	Spring doit écouter sur le port 8080
`401`	Identifiants absents ou invalides ; vérifier la branche et le compte disponible
Mutation `403` en étapes 2 et 3	CSRF par défaut ; comportement attendu sans jeton
Mutation `403` après l'exercice 4	Vérifier le TODO 4.6 et le redémarrage de Spring
Statistiques `403` avec Alice après l'exercice 4	Comportement attendu : rôle `ADMIN` requis
Note `404` en branches 1 à 4 avec un compte autorisé	Route absente, fournie seulement à partir de la branche 5
Alice lit la note après l'exercice 5	Vérifier `@PreAuthorize` et `@EnableMethodSecurity`
Tests ignorés	Retirer `@Disabled` seulement après avoir terminé l'exercice correspondant
Ancien comportement après un changement de branche	Arrêter puis relancer Spring et Vite

`401` indique une authentification absente ou invalide. `403` indique un refus de sécurité : rôle insuffisant dans le parcours final, ou jeton CSRF absent aux étapes précédentes. `404` indique une route ou une tâche absente, lorsque la sécurité autorise l'appel.

Bilan à rendre : fiche d'analyse, configurations des exercices, résultats des tests et observations dans Planora. Expliquez la distinction entre authentification et autorisation, l'intérêt de BCrypt, l'ordre des règles HTTP et l'activation de la sécurité des méthodes.

HTTP Basic encode les identifiants en Base64 ; il ne les chiffre pas. Les comptes et données de ce TP sont fictifs et destinés au local. Cette démonstration n'est pas une configuration de production.

Ressources : `ENONCE.md` pour l'exercice courant ; `docs/corrections/` pour les corrigés précédents ; `docs/ARCHITECTURE.md`, `docs/SCENARIOS_DE_TEST.md` et `docs/GUIDE_HTTP.md` pour les détails.